Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 1
з дисципліни: «Теорія алгоритмів»
Тема: «Квадратичні алгоритми сортування»

Варіант № 2

Виконав:
Студент групи АІ-254
Гармаш Г.Є.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: Метою виконання лабораторної роботи є набуття практичних навичок із проєктування, реалізації, тестування та аналізу алгоритмів сортування вибором та вставкою. За рахунок виконання трасування та тестування студент має практично довести квадратичну часову складність та стійкість вивчених алгоритмів, а також порівняти ці алгоритми за кількістю базових операцій.

Завдання:
1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм сортування вибором.
2. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм сортування вставками
3. Порівняти обчислювальну складність алгоритмів сортування вибором та вставками за базовими операціями.

2	79, 97, 82, 18, 20, 2, 88, 61, 17

Результати виконання завдання:

1. Сортування вибором

Лістинг 1.1 – Псевдокод алгоритму сортування вибором

```
Алгоритм Selection_sort (A [0..n-1])
// Сортування масиву методом вибору
// Вхідні дані: Масив A [0.. n - 1] елементів, що впорядковуються
// Вихідні дані: Масив A [0.. n - 1], відсортований у неспадному порядку
for i ← 0 to n-2 do // зовнішній цикл
{
    min ← i
    for j ← i+1 to n -1 do // внутрішній цикл
        if A[j] < A[min]
            min ← j
    B ← A[i] // Обмін A[i] та A [min]
    A[i] ← A[min]
    A[min] ← B
}
```

Лістинг 1.2 – Python-код алгоритму сортування вибором.

```
def selection_sort(arr):
    n = len(arr)
    comparisons = 0
    assignments = 0

    for i in range(n - 1):
        print(f"Ітерація {i}:")
        print(f"  Поточний елемент (для обміну): arr[{i}] = {arr[i]}")
        print(f"  Шукаємо мінімальний елемент у частині: {arr[i:]}")

        min_index = i
        assignments += 1

        for j in range(i + 1, n):
            comparisons += 1
```

```
            if arr[j] < arr[min_index]:
                min_index = j
                assignments += 1

        print(f"    Знайдено мінімальний елемент: arr[{min_index}] =
{arr[min_index]}")

        comparisons += 1
        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            assignments += 3
            print(f"  Обмін arr[{i}] ({arr[min_index]}) і arr[{min_index}]
({arr[i]})")
        else:
            print("  Обмін не потрібен, елемент на своєму місці")

        print(f"  Масив після ітерації {i}: {arr}")
        print("-" * 30)

    return arr, comparisons, assignments

my_list = [79, 97, 82, 18, 20, 2, 88, 61, 17]
print("Початковий масив:", my_list)

sorted_list, comps, assigns = selection_sort(my_list.copy())

print("Фінальний відсортований масив:", sorted_list)
print(f"Кількість порівнянь: {comps}")
print(f"Кількість присвоєнь: {assigns}")
```

Результати моделювання та трасування показано в таблиці 1.1

Таблиця 1.1 Результати чисельного моделювання та трасування алгоритму сортування вибором.

Чисельне моделювання псевдокоду Лістинг 1.1	Трасування Python-коду Лістинг 1.2
Вихідні дані A=[79_0, 97_1, 82_2, 18_3, 20_4, 2_5, 88_6, 61_7, 17_8] n=9 (for i) 1. i=0; min=0; j=0+1=1; A[1]<A[0] (97<79 - False) j=2; A[2]<A[0] (82<79 - False) j=3; A[3]<A[0] (18<79) min="3<br">j=4; A[4]<A[3] (20<18 - False) j=5; A[5]<A[3] (2<18) min="5<br">j=6; A[6]<A[5] (88<2 - False) j=7; A[7]<A[5] (61<2 - False) j=8; A[8]<A[5] (17<2 - False) (end for j) b=A[0]; A[0]=A[5]; A[5]=b; A[0]=2; A[5]=79 A=[2_0, 97_1, 82_2, 18_3, 20_4, 79_5, 88_6, 61_7, 17_8] 2. i=1; min=1; j=1+1=2; A[2]<A[1] (82<97) min="2<br">j=3; A[3]<A[2]	Початковий масив: [79, 97, 82, 18, 20, 2, 88, 61, 17] Ітерація 0: Поточний елемент (для обміну): arr[0] = 79 Шукаємо мінімальний елемент у частині: [79, 97, 82, 18, 20, 2, 88, 61, 17] Знайдено мінімальний елемент: arr[5] = 2 Обмін arr[0] (79) і arr[5] (2) Масив після ітерації 0: [2, 97, 82, 18, 20, 79, 88, 61, 17] Ітерація 1: Поточний елемент (для обміну): arr[1] = 97 Шукаємо мінімальний елемент у частині: [97, 82, 18, 20, 79, 88, 61, 17] Знайдено мінімальний елемент: arr[8] = 17

(18<82) min="3<br">j=4; A[4]<A[3] (20<18 - False) j=5; A[5]<A[3] (79<18 - False) j=6; A[6]<A[3] (88<18 - False) j=7; A[7]<A[3] (61<18 - False) j=8; A[8]<A[3] (17<18) min="8<br">(end for j) b=A[1]; A[1]=A[8]; A[8]=b; A[1]=17; A[8]=97 A=[2_0, 17_1, 82_2, 18_3, 20_4, 79_5, 88_6, 61_7, 97_8]	Обмін arr[1] (97) і arr[8] (17) Масив після ітерації 1: [2, 17, 82, 18, 20, 79, 88, 61, 97]
3. i=2; min=2; j=2+1=3; A[3]<A[2] (18<82) min="3<br">j=4; A[4]<A[3] (20<18 - False) j=5; A[5]<A[3] (79<18 - False) j=6; A[6]<A[3] (88<18 - False) j=7; A[7]<A[3] (61<18 - False) j=8; A[8]<A[3] (97<18 - False) (end for j) b=A[2]; A[2]=A[3]; A[3]=b; A[2]=18; A[3]=82 A=[2_0, 17_1, 18_2, 82_3, 20_4, 79_5, 88_6, 61_7, 97_8]	Ітерація 2: Поточний елемент (для обміну): arr[2] = 82 Шукаємо мінімальний елемент у частині: [82, 18, 20, 79, 88, 61, 97] Знайдено мінімальний елемент: arr[3] = 18 Обмін arr[2] (82) і arr[3] (18) Масив після ітерації 2: [2, 17, 18, 82, 20, 79, 88, 61, 97]
4. i=3; min=3; j=3+1=4; A[4]<A[3] (20<82) min="4<br">j=5; A[5]<A[4] (79<20 - False) j=6; A[6]<A[4] (88<20 - False) j=7; A[7]<A[4] (61<20 - False) j=8; A[8]<A[4] (97<20 - False) (end for j) b=A[3]; A[3]=A[4]; A[4]=b; A[3]=20; A[4]=82 A=[2_0, 17_1, 18_2, 20_3, 82_4, 79_5, 88_6, 61_7, 97_8]	Ітерація 3: Поточний елемент (для обміну): arr[3] = 82 Шукаємо мінімальний елемент у частині: [82, 20, 79, 88, 61, 97] Знайдено мінімальний елемент: arr[4] = 20 Обмін arr[3] (82) і arr[4] (20) Масив після ітерації 3: [2, 17, 18, 20, 82, 79, 88, 61, 97]
5. i=4; min=4; j=4+1=5; A[5]<A[4] (79<82) min="5<br">j=6; A[6]<A[5] (88<79 - False) j=7; A[7]<A[5] (61<79) min="7<br">j=8; A[8]<A[7] (97<61 - False) (end for j) b=A[4]; A[4]=A[7]; A[7]=b; A[4]=61; A[7]=82 A=[2_0, 17_1, 18_2, 20_3, 61_4, 79_5, 88_6, 82_7, 97_8]	Ітерація 4: Поточний елемент (для обміну): arr[4] = 82 Шукаємо мінімальний елемент у частині: [82, 79, 88, 61, 97] Знайдено мінімальний елемент: arr[7] = 61 Обмін arr[4] (82) і arr[7] (61) Масив після ітерації 4: [2, 17, 18, 20, 61, 79, 88, 82, 97]
6. i=5; min=5; j=5+1=6; A[6]<A[5] (88<79 - False) j=7; A[7]<A[5] (82<79 - False) j=8; A[8]<A[5] (97<79 - False) (end for j) b=A[5]; A[5]=A[5]; A[5]=b; A[5]=79; A[5]=79 A=[2_0, 17_1, 18_2, 20_3, 61_4, 79_5, 88_6, 82_7, 97_8]	Ітерація 5: Поточний елемент (для обміну): arr[5] = 79 Шукаємо мінімальний елемент у частині: [79, 88, 82, 97] Знайдено мінімальний елемент: arr[5] = 79 Обмін не потрібен, елемент на своєму місці Масив після ітерації 5: [2, 17, 18, 20, 61, 79, 88, 82, 97]
	Ітерація 6:

7. i=6; min=6; j=6+1=7; A[7]<A[6] (82<88) min="7<br">j=8; A[8]<A[7] (97<82 - False) (end for j) b=A[6]; A[6]=A[7]; A[7]=b; A[6]=82; A[7]=88 A=[2_0, 17_1, 18_2, 20_3, 61_4, 79_5, 82_6, 88_7, 97_8]	Поточний елемент (для обміну): arr[6] = 88 Шукаємо мінімальний елемент у частині: [88, 82, 97] Знайдено мінімальний елемент: arr[7] = 82 Обмін arr[6] (88) і arr[7] (82) Масив після ітерації 6: [2, 17, 18, 20, 61, 79, 82, 88, 97]
8. i=7; min=7; j=7+1=8; A[8]<A[7] (97<88 - False) (end for j) b=A[7]; A[7]=A[7]; A[7]=b; A[7]=88; A[7]=88 A=[2_0, 17_1, 18_2, 20_3, 61_4, 79_5, 82_6, 88_7, 97_8] (end for i)	Ітерація 7:Поточний елемент (для обміну): arr[7] = 88 Шукаємо мінімальний елемент у частині: [88, 97] Знайдено мінімальний елемент: arr[7] = 88 Обмін не потрібен, елемент на своєму місці Масив після ітерації 7: [2, 17, 18, 20, 61, 79, 82, 88, 97] Сортування завершено. Фінальний відсортований масив: [2, 17, 18, 20, 61, 79, 82, 88, 97] Загальна кількість порівнянь: 44 Загальна кількість присвоєнь: 35

2. Сортування вставками

Лістинг 1.3 – Псевдокод алгоритму сортування вставками.

```
Insertion Sort
for j = 2 to A.length do
    key = A[j]
    i = j - 1
    while (int i > 0 and A[i] > key) do
        A[i+1] = A[i]
        i = i - 1
    end while
    A[i+1] = key
end
```

Лістинг 1.4 – Python-код алгоритму сортування вставками.

```
def insertion_sort(arr):
    n = len(arr)
    comparisons = 0
    assignments = 0

    for i in range(1, n):
        key = arr[i]
        assignments += 1
        j = i - 1
        assignments += 1

        print(f"Ітерація {i}:")
        print(f"  Елемент для вставки (key): {key}")
```

```
        print(f"  Відсортована частина: {arr[:i]}")

        while j >= 0 and arr[j] > key:
            comparisons += 1
            print(f"  Порівняння: {arr[j]} > {key} True. Зсуваємо {arr[j]}
вправо.")
            arr[j + 1] = arr[j]
            assignments += 1
            j -= 1
            assignments += 1

        if j >= 0:
            comparisons += 1
            print(f"  Порівняння: {arr[j]} > {key} False. Цикл завершено.")
        else:
            print("  Досягнуто початку масиву. Цикл завершено.")

        arr[j + 1] = key
        assignments += 1
        print(f"  Вставка {key} на позицію {j + 1}.")
        print(f"  Масив після ітерації {i}: {arr}")
        print("-" * 30)

    return arr, comparisons, assignments

my_list = [79, 97, 82, 18, 20, 2, 88, 61, 17]
print("Початковий масив:", my_list)
print("-" * 30)

sorted_list, comps, assigs = insertion_sort(my_list.copy())

print("Фінальний відсортований масив:", sorted_list)
print(f"Загальна кількість порівнянь: {comps}")
print(f"Загальна кількість присвоєнь: {assigs}")
```

Таблиця 1.2 Результати чисельного моделювання та трасування алгоритму сортування вставками.

Чисельне моделювання псевдокоду Лістинг 1.3	Трасування Python-коду Лістинг 1.4
Вихідні дані A=[79_1, 97_2, 82_3, 18_4, 20_5, 2_6, 88_7, 61_8, 17_9] n=9 (for j) 1. j=2; key=A[2]=97; i=2-1=1; i>0 (True) A[1]>key (79>97 - False); A[2]=key=97 A=[79_1, 97_2, 82_3, 18_4, 20_5, 2_6, 88_7, 61_8, 17_9] 2. j=3; key=A[3]=82; i=3-1=2; i>0 (True) A[2]>key (97>82 - True) → A[3]=A[2]=97; A=[79_1, 97_2, 97_3, 18_4, 20_5, 2_6, 88_7, 61_8, 17_9] i=2-1=1; i>0 (True) A[1]>key (79>82 - False); A[2]=key=82 A=[79_1, 82_2, 97_3, 18_4, 20_5, 2_6, 88_7, 61_8, 17_9] 3. j=4; key=A[4]=18; i=4-1=3;	Початковий масив: [79, 97, 82, 18, 20, 2, 88, 61, 17] Ітерація 1: Елемнт для вставки (key): 97 Відсортована частина: [79] Порівняння: 79 > 97 False. Цикл завершено. Вставка 97 на позицію 1. Масив після ітерації 1: [79, 97, 82, 18, 20, 2, 88, 61, 17] Ітерація 2: Елемент для вставки (key): 82 Відсортована частина: [79, 97] Порівняння: 97 > 82 True. Зсуваємо 97 вправо. Порівняння: 79 > 82 False. Цикл завершено.

i>0 (True) A[3]>key (97>18 - True) → A[4]=A[3]=97;
i=3-1=2;
i>0 (True) A[2]>key (82>18 - True) → A[3]=A[2]=82;
i=2-1=1;
i>0 (True) A[1]>key (79>18 - True) → A[2]=A[1]=79;
i=1-1=0; i>0 (False)
A[1]=key=18
A=[18_1, 79_2, 82_3, 97_4, 20_5, 2_6, 88_7, 61_8, 17_9]
	Вставка 82 на позицію 1.
	Масив після ітерації 2:
	[79, 82, 97, 18, 20, 2, 88, 61, 17]

4. j=5; key=A[5]=20; i=5-1=4;
i>0 (True) A[4]>key (97>20 - True) → A[5]=A[4]=97;
i=4-1=3;
i>0 (True) A[3]>key (82>20 - True) → A[4]=A[3]=82;
i=3-1=2;
i>0 (True) A[2]>key (79>20 - True) → A[3]=A[2]=79;
i=2-1=1;
i>0 (True) A[1]>key (18>20 - False);
A[2]=key=20
A=[18_1, 20_2, 79_3, 82_4, 97_5, 2_6, 88_7, 61_8, 17_9]
	Ітерація 3:
	Елемент для вставки (key): 18
	Відсортована частина: [79, 82, 97]
	Порівняння: 97 > 18 True. Зсуваємо 97 вправо.
	Порівняння: 82 > 18 True. Зсуваємо 82 вправо.
	Порівняння: 79 > 18 True. Зсуваємо 79 вправо.
	Досягнуто початку масиву. Цикл завершено.
	Вставка 18 на позицію 0.
	Масив після ітерації 3:
	[18, 79, 82, 97, 20, 2, 88, 61, 17]

5. j=6; key=A[6]=2; i=6-1=5; i>0 (True) A[5]>key (97>2 - True) → A[6]=A[5]=97; ... (зсув 82, 79, 20, 18) i=0; i>0 (False) A[1]=key=2 A=[2_1, 18_2, 20_3, 79_4, 82_5, 97_6, 88_7, 61_8, 17_9]	Ітерація 4: Елемент для вставки (key): 20 Відсортована частина: [18, 79, 82, 97] Порівняння: 97 > 20 True. Зсуваємо 97 вправо. Порівняння: 82 > 20 True. Зсуваємо 82 вправо. Порівняння: 79 > 20 True. Зсуваємо 79 вправо. Порівняння: 18 > 20 False. Цикл завершено. Вставка 20 на позицію 1. Масив після ітерації 4: [18, 20, 79, 82, 97, 2, 88, 61, 17]
6. j=7; key=A[7]=88; i=7-1=6; i>0 (True) A[6]>key (97>88 - True) → A[7]=A[6]=97; i=6-1=5; i>0 (True) A[5]>key (82>88 - False); A[6]=key=88 A=[2_1, 18_2, 20_3, 79_4, 82_5, 88_6, 97_7, 61_8, 17_9]	Ітерація 5: Елемент для вставки (key): 2 Відсортована частина: [18, 20, 79, 82, 97] Порівняння: 97 > 2 True. Зсуваємо 97 вправо. Порівняння: 82 > 2 True. Зсуваємо 82 вправо. Порівняння: 79 > 2 True. Зсуваємо 79 вправо. Порівняння: 20 > 2 True. Зсуваємо 20 вправо. Порівняння: 18 > 2 True. Зсуваємо 18 вправо. Досягнуто початку масиву. Цикл завершено. Вставка 2 на позицію 0. Масив після ітерації 5: [2, 18, 20, 79, 82, 97, 88, 61, 17]
7. j=8; key=A[8]=61; i=8-1=7; A[7]>key (97>61 - True) → A[8]=A[7]=97; A[6]>key (88>61 - True) → A[7]=A[6]=88; A[5]>key (82>61 - True) → A[6]=A[5]=82; A[4]>key (79>61 - True) → A[5]=A[4]=79; A[3]>key (20>61 - False); A[4]=key=61	

A=[2_1, 18_2, 20_3, 61_4, 79_5, 82_6, 88_7, 97_8, 17_9] 8. j=9; key=A[9]=17; i=9-1=8; A[8]>17 (97>17 - True) → A[9]=97; ... (зсув 88, 82, 79, 61, 20, 18) A[1]>17 (2>17 - False); A[2]=key=17 A=[2_1, 17_2, 18_3, 20_4, 61_5, 79_6, 82_7, 88_8, 97_9] (End for j)	Ітерація 6: Елемент для вставки (key): 88 Відсортована частина: [2, 18, 20, 79, 82, 97] Порівняння: 97 > 88 True. Зсуваємо 97 вправо. Порівняння: 82 > 88 False. Цикл завершено. Вставка 88 на позицію 5. Масив після ітерації 6: [2, 18, 20, 79, 82, 88, 97, 61, 17] Ітерація 7: Елемент для вставки (key): 61 Відсортована частина: [2, 18, 20, 79, 82, 88, 97] Порівняння: 97 > 61 True. Зсуваємо 97 вправо. Порівняння: 88 > 61 True. Зсуваємо 88 вправо. Порівняння: 82 > 61 True. Зсуваємо 82 вправо. Порівняння: 79 > 61 True. Зсуваємо 79 вправо. Порівняння: 20 > 61 False. Цикл завершено. Вставка 61 на позицію 3. Масив після ітерації 7: [2, 18, 20, 61, 79, 82, 88, 97, 17] Ітерація 8: Елемент для вставки (key): 17 Відсортована частина: [2, 18, 20, 61, 79, 82, 88, 97] Порівняння: 97 > 17 True. Зсув 97 вправо. ... Порівняння: 18 > 17 True. Зсув 18 вправо. Порівняння: 2 > 17 False. Цикл завершено. Вставка 17 на позицію 1. Масив після ітерації 8: [2, 17, 18, 20, 61, 79, 82, 88, 97] Сортування завершено. Фінальний відсортований масив: [2, 17, 18, 20, 61, 79, 82, 88, 97] Загальна кількість порівнянь: 30 Загальна кількість присвоєнь: 72

3. Аналіз обчислювальної складності

Таблиця 1.3 Порівняння обчислювальної складності алгоритмів сортування вибором та вставками

Назва алгоритму	Теоретичні відомості		Результати експерименту n=9 A=[79_0, 97_1, 82_2, 18_3, 20_4, 2_5, 88_6, 61_7, 17_8]	
	порівняння	присвоювання	порівняння	присвоювання
Selection Sort	$O(n^2)$,	$O(n)$	44	35
Insertion Sort	$O(n)$ *(Best)* $O(n^2)$ *(Worst, Average)*	$O(n)$ *(Best)* $O(n^2)$ *(Worst, Average)*	30	72

Висновки: Під час виконання першої лабораторної роботи я набув практичних навичок із проєктування, реалізації, тестування та аналізу квадратичних алгоритмів сортування: вибором (Selection sort) та вставками (Insertion sort). Було розроблено програмний код мовою Python, виконано ручне чисельне моделювання та трасування алгоритмів для індивідуального масиву варіанту №2. За рахунок підрахунку базових операцій (порівнянь та присвоювань) було практично досліджено квадратичну часову складність $O(n^2)$, цих алгоритмів. Порівняльний аналіз показав, що для моєї вхідної послідовності алгоритм сортування вибором виконав більше порівнянь (44 проти 30), але значно менше операцій присвоювання (35 проти 72 у сортування вставками). Це підтверджує теоретичні відомості: сортування вибором мінімізує кількість обмінів пам'яті $O(n)$, тоді як сортування вставками ефективніше за кількістю порівнянь, але вимагає великої кількості зсувів (присвоювань) для вставки елементів на потрібні місця.

Посилання на GitHub: https://github.com/9998920/algorithms-labs